Lab C Bonus B

```
Here, it is assumed that you have your own URL. I'm using che
wbacca-growl.com as an example


Alright — Lab 1C-Bonus-B (and yes, we'll assume the students
own chewbacca-growl.com).
This turns your stack into a real enterprise pattern:
  Public ALB (internet-facing)
  Private EC2 targets (no public IP)
  TLS with ACM for chewbacca-growl.com
  WAF attached to ALB
  CloudWatch Dashboard
  SNS alarm on ALB 5xx spikes

This is exactly how modern companies ship: IaC + private comp
ute + managed ingress + TLS + WAF + monitoring + paging.
If you can submit this in Terraform, you're no longer "a stud
ent who clicked around"— you're a junior cloud engineers.

Below is a Terraform skeleton overlay you can add to your exi
sting 1C + Bonus-A repo.

Add 1c_bonus_variables.tf (append to variables.tf)

Add file: bonus_b.tf
This assumes you already have from 1C / Bonus-A:
    aws_vpc.chewbacca_vpc01
    aws_subnet.chewbacca_public_subnets
    aws_subnet.chewbacca_private_subnets
    aws_security_group.chewbacca_ec2_sg01 (for private EC2)
    aws_instance.chewbacca_ec201_private_bonus (private EC2 i
nstance)
```

```
    aws_sns_topic.chewbacca_sns_topic01 (SNS topic)

Add file: Bonus-B_outputs.tf

What you must implement (so you learn the right pain)
TLS (ACM) validation for app.chewbacca-growl.com
You gave them the domain; they must complete one of:
  DNS validation (best): create Route53 hosted zone + validat
ion records in Terraform
  Email validation (acceptable): do it manually, then Terrafo
rm continues (less ideal)

Suggested student path (DNS):
  Create Route53 Hosted Zone for chewbacca-growl.com
  Add aws_route53_record for ACM validation
  Add a CNAME (or ALIAS) pointing app.chewbacca-growl.com → A
LB DNS

  I didn’t auto-add Route53 resources because some students m
ay manage DNS outside Route53.
  But if you want, I can provide a Route53 skeleton too (Host
ed Zone + records + ACM validation),
  Chewbacca-style.

ALB SG rules
you must add:
  inbound 80/443 from 0.0.0.0/0
  outbound to targets on app port

EC2 runs app on the target port
They must ensure their user-data/app listens on port 80 (or u
pdate TG/SG accordingly).

Verification commands (CLI) for Bonus-B
1) ALB exists and is active
```

```
      aws elbv2 describe-load-balancers \
        --names chewbacca-alb01 \
        --query "LoadBalancers[0].State.Code"

3) HTTPS listener exists on 443

      aws elbv2 describe-listeners \
        --load-balancer-arn <ALB_ARN> \
        --query "Listeners[].Port"

4) Target is healthy

      aws elbv2 describe-target-health \
        --target-group-arn <TG_ARN>

5) WAF attached

      aws wafv2 get-web-acl-for-resource \
        --resource-arn <ALB_ARN>

7) Alarm created (ALB 5xx)

      aws cloudwatch describe-alarms \
        --alarm-name-prefix chewbacca-alb-5xx

9) Dashboard exists

      aws cloudwatch list-dashboards \
        --dashboard-name-prefix chewbacca
```

Bonus-B_outputs.tf

bonus_b.tf

bonus_b_logging_route53_apex.tf

bonus_b_route53.tf

bonus_b_waf_logging.tf

Lab C Bonus B: Execution Plan

We are upgrading your stack to an enterprise-grade architecture with a Public ALB, Private EC2 targets, TLS (HTTPS) for `williebright.com`, WAF protection, and CloudWatch monitoring.

Here are the step-by-step instructions to execute this lab efficiently.

Step 1: Define New Variables

Create a new file named `1c_bonus_variables.tf` (or append to your existing `variables.tf`) to define the new variables required for the ALB, WAF, and Domain.

Copy this content:

```
variable "domain_name" {
  type    = string
  default = "williebright.com"
}

variable "app_subdomain" {
  type    = string
  default = "app" # This makes your app accessible at app.wil
liebright.com
}

variable "certificate_validation_method" {
  type    = string
  default = "DNS"
}

variable "enable_waf" {
  type    = bool
  default = true
}
```

```
variable "manage_route53_in_terraform" {
  type        = bool
  default     = true
  description = "Set to true to create the Hosted Zone via Te
rraform. Set to false if you already have a Hosted Zone ID."
}

variable "route53_hosted_zone_id" {
  type        = string
  default     = "" # Only required if manage_route53_in_terra
form is false
  description = "The ID of your existing Route53 Hosted Zone"
}

variable "alb_5xx_evaluation_periods" {
  type    = number
  default = 1
}

variable "alb_5xx_threshold" {
  type    = number
  default = 0
}

variable "alb_5xx_period_seconds" {
  type    = number
  default = 60
}

variable "enable_alb_access_logs" {
  type    = bool
  default = true
}

variable "alb_access_logs_prefix" {
```

```
  type    = string
  default = "alb-logs"
}

variable "waf_log_destination" {
  type    = string
  default = "cloudwatch" # Options: cloudwatch, s3, firehose
}

variable "waf_log_retention_days" {
  type    = number
  default = 30
}
```

Step 2: Create the Main Bonus Logic (`bonus_b.tf`)

Create a file named `bonus_b.tf`. This contains the ALB, Security Groups, WAF, and CloudWatch resources.

Important: You must fill in the `TODO` sections (Security Group rules).

1. **ALB Security Group:** needs inbound port 80 and 443 from `0.0.0.0/0`.
2. **Target Group Attachment:** needs to reference your **Private EC2** instance ID from Lab 1C/Bonus A.

```
locals {
  chewbacca_fqdn = "${var.app_subdomain}.${var.domain_name}"
}

# --- 1. Security Groups ---

resource "aws_security_group" "chewbacca_alb_sg01" {
  name        = "${var.project_name}-alb-sg01"
  description = "ALB security group"
  vpc_id      = aws_vpc.chewbacca_vpc01.id
```

```
  # Inbound HTTP
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # Inbound HTTPS
  ingress {
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # Outbound to everywhere (for health checks/forwarding)
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = { Name = "${var.project_name}-alb-sg01" }
}

# Allow ALB to reach EC2
resource "aws_security_group_rule" "chewbacca_ec2_ingress_fro
m_alb01" {
  type                     = "ingress"
  security_group_id        = aws_security_group.chewbacca_ec2
_sg01.id # Ensure this matches your EC2 SG name
  from_port                = 80
  to_port                  = 80
  protocol                 = "tcp"
```

```
  source_security_group_id = aws_security_group.chewbacca_alb
_sg01.id
}

# --- 2. ALB & Target Group ---

resource "aws_lb" "chewbacca_alb01" {
  name               = "${var.project_name}-alb01"
  load_balancer_type = "application"
  internal           = false
  security_groups    = [aws_security_group.chewbacca_alb_sg0
1.id]
  subnets            = aws_subnet.chewbacca_public_subnets
[*].id
  tags               = { Name = "${var.project_name}-alb01" }
}

resource "aws_lb_target_group" "chewbacca_tg01" {
  name     = "${var.project_name}-tg01"
  port     = 80
  protocol = "HTTP"
  vpc_id   = aws_vpc.chewbacca_vpc01.id

  health_check {
    enabled             = true
    path                = "/" # Change to /health or /list if
needed
    matcher             = "200-399"
  }
}

resource "aws_lb_target_group_attachment" "chewbacca_tg_attac
h01" {
  target_group_arn = aws_lb_target_group.chewbacca_tg01.arn
  target_id        = aws_instance.chewbacca_ec201_private_bon
us.id # Ensure this matches your Private EC2 resource name
```

```
  port              = 80
}

# --- 3. TLS Certificate ---

resource "aws_acm_certificate" "chewbacca_acm_cert01" {
  domain_name       = local.chewbacca_fqdn
  validation_method = var.certificate_validation_method
  tags              = { Name = "${var.project_name}-acm-cert0
1" }
}

resource "aws_acm_certificate_validation" "chewbacca_acm_vali
dation01" {
  certificate_arn = aws_acm_certificate.chewbacca_acm_cert01.
arn
  # We will define the validation records in the Route53 file
}

# --- 4. Listeners ---

resource "aws_lb_listener" "chewbacca_http_listener01" {
  load_balancer_arn = aws_lb.chewbacca_alb01.arn
  port              = 80
  protocol          = "HTTP"

  default_action {
    type = "redirect"
    redirect {
      port        = "443"
      protocol    = "HTTPS"
      status_code = "HTTP_301"
    }
  }
}
```

```
resource "aws_lb_listener" "chewbacca_https_listener01" {
  load_balancer_arn = aws_lb.chewbacca_alb01.arn
  port              = 443
  protocol          = "HTTPS"
  ssl_policy        = "ELBSecurityPolicy-TLS13-1-2-2021-06"
  certificate_arn   = aws_acm_certificate.chewbacca_acm_cert0
1.arn

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.chewbacca_tg01.arn
  }
}
```

Step 3: Configure Route53 (`bonus_b_route53.tf`)

This handles your domain `williebright.com` and validates the certificate.

```
locals {
  chewbacca_zone_id = var.manage_route53_in_terraform ? aws_r
oute53_zone.chewbacca_zone01[0].zone_id : var.route53_hosted_
zone_id
}

# Create Hosted Zone (if managed by TF)
resource "aws_route53_zone" "chewbacca_zone01" {
  count = var.manage_route53_in_terraform ? 1 : 0
  name  = var.domain_name
  tags  = { Name = "${var.project_name}-zone01" }
}

# DNS Validation Records
resource "aws_route53_record" "chewbacca_acm_validation_recor
ds01" {
  for_each = {
    for dvo in aws_acm_certificate.chewbacca_acm_cert01.domai
```

```
n_validation_options : dvo.domain_name => {
      name   = dvo.resource_record_name
      type   = dvo.resource_record_type
      record = dvo.resource_record_value
    }
  }
  zone_id = local.chewbacca_zone_id
  name    = each.value.name
  type    = each.value.type
  ttl     = 60
  records = [each.value.record]
}

# Alias Record (app.williebright.com -> ALB)
resource "aws_route53_record" "chewbacca_app_alias" {
  zone_id = local.chewbacca_zone_id
  name    = "${var.app_subdomain}.${var.domain_name}"
  type    = "A"

  alias {
    name                   = aws_lb.chewbacca_alb01.dns_name
    zone_id                = aws_lb.chewbacca_alb01.zone_id
    evaluate_target_health = true
  }
}
```

Step 4: Add Outputs (`Bonus-B_` `outputs.tf`)

```
output "chewbacca_alb_dns_name" {
  value = aws_lb.chewbacca_alb01.dns_name
}

output "chewbacca_app_fqdn" {
```

```
  value = "${var.app_subdomain}.${var.domain_name}"
}
```

Step 5: Apply & Verify

1. Run `terraform init` to pick up new providers if needed.
2. Run `terraform apply`.
3. **Wait for Certificate Validation:** It may take a few minutes for the DNS records to propagate and ACM to validate your certificate.
4. **Verify via CLI:**

```
# 1. Check ALB State
aws elbv2 describe-load-balancers --names chewbacca-alb01 --q
uery "LoadBalancers[0].State.Code"

# 2. Check Target Health (Should be healthy)
aws elbv2 describe-target-health --target-group-arn <Your-Tar
get-Group-ARN>

# 3. Test HTTPS (Should work!)
curl -v https://app.williebright.com
```

sudo systemctl restart rdsapp

Important Notes

- **Names:** I used `chewbacca` as the resource prefix to match the lab skeleton. If your existing resources (VPC, Subnets) use a different prefix (e.g., `willie` or `lab1c`), ensure you update the references in the code above (e.g., `aws_vpc.chewbacca_vpc01.id` → `aws_vpc.your_vpc_name.id`).
- **WAF:** I omitted the full WAF configuration for brevity, but you can enable it by ensuring `var.enable_waf = true` and adding the `aws_wafv2_web_acl` resource from the lab files if desired.

- **Domain:** Ensure your domain registrar for `williebright.com` points to the Name Servers of the Route53 Hosted Zone created by Terraform (you'll see them in the AWS Console under Route53 after `apply`).

The context that you had was off.